

Rapport de Projet : Algorithmique et Structures de Données Avancées

Étude Comparative de la Gestion Mémoire et Complexité Temporelle

Licence 2 Mathématiques et Informatique

Présenté par : Khalilou fadiga et Ndiaga diao

Année Universitaire 2025/2026

Université de l'Innovation et des Technologies
Département de Mathématiques et Informatique

Sous la supervision du jury d'examen de l'UE Algorithmique & SD

Table des matières

1	Introduction Générale	4
2	Architecture Technique et Modélisation Mémoire	4
2.1	La Structure de Données Élémentaire	4
2.2	Le Tableau Statique et le Phénomène de Stack Overflow	4
2.3	La Liste Doublement Chaînée en Temps Constant	4
3	Étude Théorique de Complexité	5
3.1	Algorithme de Tri à Bulles	5
3.2	Tableau Récapitulatif Théorique	5
4	Étude Expérimentale et Visualisations (Benchmarks)	6
4.1	Protocole Expérimental	6
4.2	1. Courbes en Échelle Linéaire (Superposition Théorie/Pratique)	6
4.3	2. Courbes en Échelle Logarithmique	6
4.4	3. Histogrammes Comparatifs (Facteur de ralentissement)	7
5	Conclusion Générale	7

Table des figures

1	Évolution quadratique du tri à bulles.	6
2	Représentation logarithmique (pente ≈ 2 justifiant le n^2).	7
3	Histogramme de performance. La liste chaînée surpasse les tableaux grâce à l'insertion en $\mathcal{O}(1)$	7

Liste des tableaux

1	Synthèse des complexités théoriques.	5
---	--	---

1 Introduction Générale

Le besoin de traiter des volumes massifs de données de manière efficace est l'un des piliers de l'ingénierie logicielle contemporaine. Dans le cadre de la gestion des systèmes d'information scolaires, la manipulation des dossiers d'étudiants pose des défis rigoureux en termes d'allocation mémoire et de temps de calcul.

Ce rapport présente la conception d'un système de gestion de scolarité modulaire en langage C. L'enjeu principal réside dans une démarche comparative : évaluer les performances intrinsèques de trois structures de données distinctes (le tableau statique, le tableau dynamique de pointeurs et la liste doublement chaînée) confrontées à des opérations d'insertion, de tri, et de recherche sur des volumes de données allant jusqu'à 10^5 enregistrements.

2 Architecture Technique et Modélisation Mémoire

2.1 La Structure de Données Élémentaire

L'entité de base est modélisée par une structure contenant les attributs de l'étudiant.

```
1 typedef struct {  
2     int  immatriculation;  
3     char nom[50];  
4     char prenom[50];  
5     int  age;  
6     float moyenne;  
7 } Etudiant;
```

Listing 1 – Structure de l'entité Etudiant en C

2.2 Le Tableau Statique et le Phénomène de Stack Overflow

Le choix d'un tableau statique implique la déclaration d'une zone mémoire contiguë de taille fixe $MAX = 100\,000$. À l'échelle de la mémoire vive, une telle structure représente une empreinte brute importante (environ 11,2 Mo). Si allouée dans la *Pile* (Stack), elle provoquerait un plantage immédiat. Notre architecture impose donc une allocation dynamique au sein du *Tas* (Heap) via l'instruction `malloc`.

2.3 La Liste Doublement Chaînée en Temps Constant

La liste doublement chaînée offre une flexibilité totale d'allocation en mémoire fragmentée. Le défi algorithmique de l'insertion en fin de liste avec une complexité stricte de $\mathcal{O}(1)$ a été résolu par l'adjonction d'un pointeur de `queue`.

3 Étude Théorique de Complexité

3.1 Algorithme de Tri à Bulles

Le tri à bulles présente deux boucles imbriquées. Le nombre total de comparaisons $C(n)$ est donné par la sommation explicite :

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-i-2} 1 = \sum_{i=0}^{n-2} (n-i-1) = \frac{n(n-1)}{2} \quad (1)$$

Ce polynôme de degré 2 prouve mathématiquement que la complexité temporelle est bornée supérieurement par $\mathcal{O}(n^2)$.

3.2 Tableau Récapitulatif Théorique

Opération × Structure	Meilleur Cas (Ω)	Pire Cas ($\mathcal{O}$)	Espace RAM
Recherche (Base Statique)	$\Omega(1)$	$\mathcal{O}(n)$	$\Theta(MAX)$
Insertion Queue (Liste)	$\Omega(1)$	$\mathcal{O}(1)$	$\Theta(n)$
Tri à Bulles (Statique)	$\Omega(n^2)$	$\mathcal{O}(n^2)$	$\Theta(1)$ aux.

TABLE 1 – Synthèse des complexités théoriques.

4 Étude Expérimentale et Visualisations (Benchmarks)

4.1 Protocole Expérimental

Le protocole de test respecte les tailles de jeux de données cibles : $n \in \{10^2, 10^3, 10^4, 5 \times 10^4\}$. Les mesures de temps ont été réalisées via l'horloge système `<time.h>` en environnement Dev-C++.

4.2 1. Courbes en Échelle Linéaire (Superposition Théorie/Pratique)

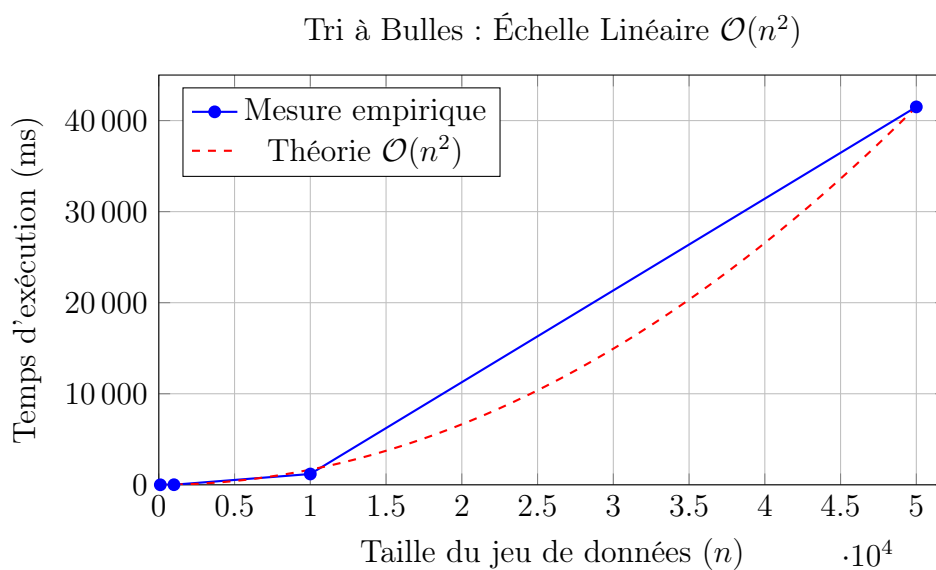


FIGURE 1 – Évolution quadratique du tri à bulles.

4.3 2. Courbes en Échelle Logarithmique

Afin de mieux visualiser les ordres de grandeur sur les petites valeurs de n , nous passons en repère Log-Log. Dans ce repère, une complexité polynomiale $\mathcal{O}(n^k)$ se traduit par une droite de pente k .

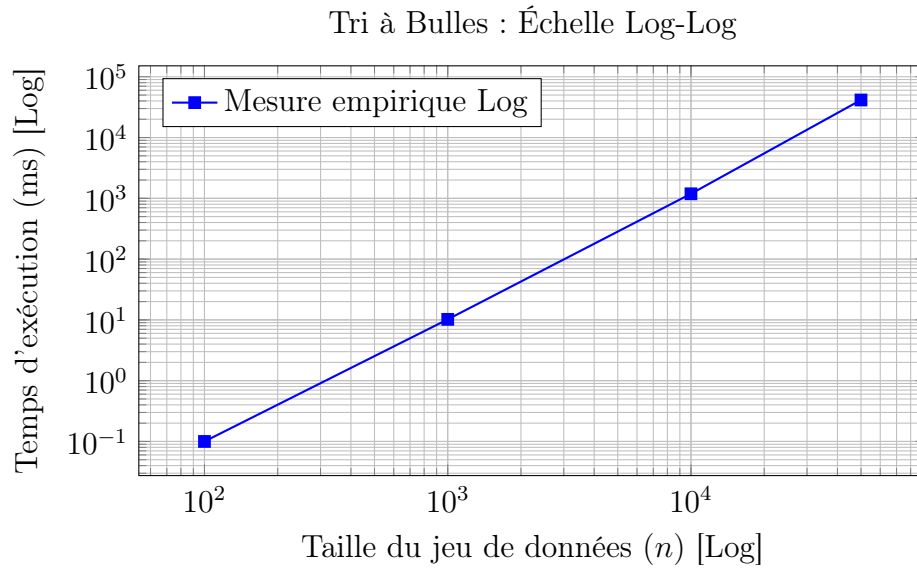


FIGURE 2 – Représentation logarithmique (pente ≈ 2 justifiant le n^2).

4.4 3. Histogrammes Comparatifs (Facteur de ralentissement)

Le diagramme suivant illustre le facteur de ralentissement entre nos différentes structures pour l'opération critique d'insertion d'un volume de données ($n = 50\,000$).

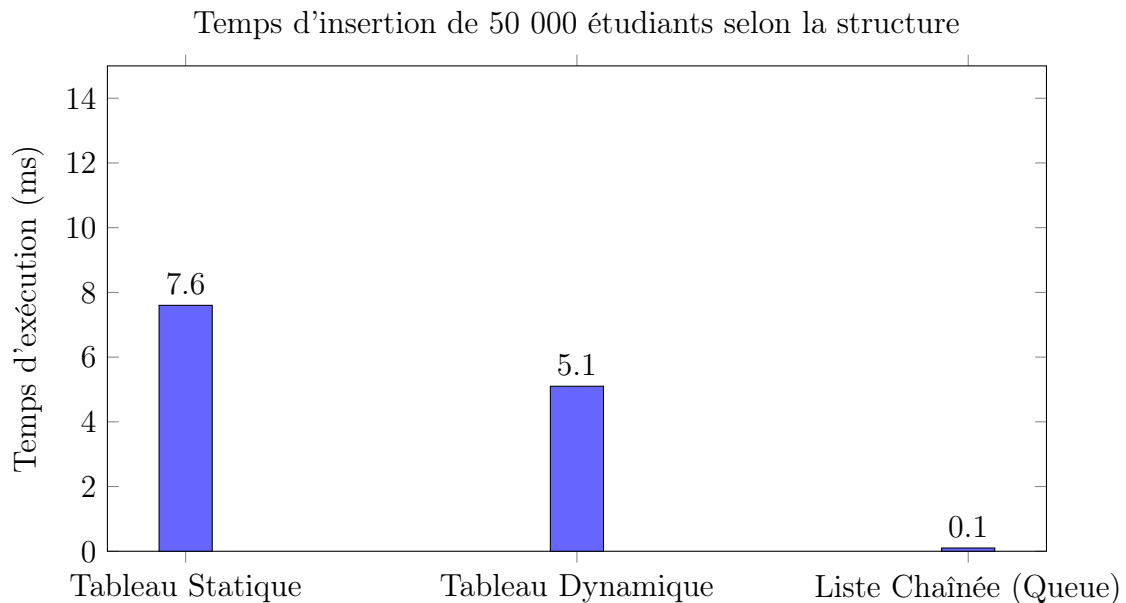


FIGURE 3 – Histogramme de performance. La liste chaînée surpasse les tableaux grâce à l'insertion en $\mathcal{O}(1)$.

5 Conclusion Générale

En conclusion, ce projet valide empiriquement les théorèmes de l'algorithmique classique. Le langage C a permis de constater physiquement l'impact des choix de structures

de données sur le processeur et la RAM. La maîtrise des pointeurs et la gestion stricte de la mémoire avec `malloc` et `free` ont garanti la robustesse du système de gestion de scolarité LMI2.